

Linear Diophantine Equations Over Polynomials and Soft Decoding of Reed–Solomon Codes

Michael Alekhovich

Abstract—This paper generalizes the classical Knuth–Schönhage algorithm computing the greatest common divisor (gcd) of two polynomials for solving arbitrary linear Diophantine systems over polynomials in time, quasi-linear in the maximal degree. As an application, the following weighted curve fitting problem is considered: given a set of points in the plane, find an algebraic curve (satisfying certain degree conditions) that goes through each point the prescribed number of times. The main motivation for this problem comes from coding theory, namely, it is ultimately related to the list decoding of Reed–Solomon codes.

This paper presents a new fast algorithm for the weighted curve fitting problem, based on the explicit construction of a Groebner basis. This gives another fast algorithm for the soft decoding of Reed–Solomon codes different from the procedure proposed by Feng, which works in time $(\frac{w}{r})^{O(1)} n \log^2 n$, where r is the rate of the code, and w is the maximal weight assigned to a vertical line.

Index Terms—Knuth–Schönhage algorithm, list decoding, Reed–Solomon codes.

I. PRELIMINARIES

LET $\mathbf{F}$ be an arbitrary field. This paper considers the following optimization problem.

Problem 1:

INPUT: An $m \times \ell$ matrix P consisting of polynomials $P_{ij} \in \mathbf{F}[x]$.

SOLUTION: a tuple of polynomials $f_1, \dots, f_\ell \in \mathbf{F}[x]$ and integers $d_1, \dots, d_m$ such that

$$\begin{aligned} \deg(P_{11}f_1 + P_{12}f_2 + \dots + P_{1\ell}f_\ell) &= d_1 \\ \deg(P_{21}f_1 + P_{22}f_2 + \dots + P_{2\ell}f_\ell) &= d_2 \\ &\vdots \\ \deg(P_{m1}f_1 + P_{m2}f_2 + \dots + P_{m\ell}f_\ell) &= d_m. \end{aligned} \quad (1)$$

GOAL:

$$\begin{aligned} \max(d_1, d_2, \dots, d_m) &\rightarrow \min \\ \max(d_1, d_2, \dots, d_m) &\neq 0. \end{aligned}$$

One can see that this problem is indeed a generalization of the greatest common divisor of two polynomials P_1 and P_2 as the latter can be expressed as

$$\deg(P_1f_1 + P_2f_2) \rightarrow \min$$

Manuscript received February 10, 2004; revised October 20, 2004. The material in this paper was presented in part at the 43rd Symposium on Foundations of Computer Science (FOCS 2002), Vancouver, BC, Canada, November 2002.

The author is with the Institute for Advanced Study, Princeton, NJ 08540 USA.

Communicated by R. J. McEliece, Associate Editor for Coding Theory.

Digital Object Identifier 10.1109/TIT.2005.850097

$$P_1f_1 + P_2f_2 \neq 0. \quad (2)$$

If f_1, f_2 is the solution of (2) then $P_1f_1 + P_2f_2$ is the gcd of P_1 and P_2 . Increasing the number of polynomials involved, one can express various problems of simultaneous approximation: given polynomials $P_1, P_2, \dots, P_m$ find $g_1, \dots, g_m, g$ of the smallest degree such that

$$g_1P_1 + g_2P_2 + \dots + g_mP_m = g.$$

The main suggested application, however, concerns algebraic interpolation problems which we will discuss later. Meanwhile, let us focus on the complexity of finding the optimal solution. It is easy to see that the problem (1) can be stated as a sequence of (ordinary) homogeneous linear systems over the coefficients of $f_1, \dots, f_\ell$ and thus can be solved in polynomial time by the Gaussian procedure. However, in the case when the degree of P_{ij} is large with respect to m and ℓ , we show a faster way to deal with this problem. The following result is a generalization of the fast Knuth–Schönhage algorithm for gcd of two polynomials (see, for example, [1] or [3, Ch. 3]). In what follows, we denote by $M(n)$ the minimal number of arithmetic operations necessary to multiply two polynomials of degree n . If the ground field supports fast Fourier transform then $M(n) \leq n \log n$, otherwise $M(n) \leq n \log n \log \log n$ (see [3, p. 32 and pp. 34–38]).

Theorem 1.1: There exists an algorithm that solves Problem 1 in time $(\ell m)^{O(1)} M(n) \log n$, where n is the maximal degree of P_{ij} .

In the second part of the paper we show an application of this result, which concerns list decoding of Reed–Solomon codes. In general, list decoding evolves around the following basic question: what information can be learned from the corrupted codeword, when the number of possible errors is too big for the unique decoding? In this case, one can hope to generate a small “list” of candidates for being the correct “transmitted” codeword. These ideas lead to an interesting and rich theory of list decoding, which has been rapidly developed during the recent years. We do not give any further general information on list decoding and confine ourselves with one particular problem, however, we advise the interested reader to see [7] for an excellent survey on the subject. The (weighted) list decoding of Reed–Solomon codes can be described as follows.

Problem 2 (Weighted Polynomial Reconstruction):

INPUT: N points $(x_1, y_1), \dots, (x_N, y_N)$, with natural weights $\text{wt}(x_1, y_1), \dots, \text{wt}(x_N, y_N)$ and integer parameters k and W .

OUTPUT: All polynomials p of degree at most k s.t.

$$\sum_{i: p(x_i) = y_i} \text{wt}(x_i, y_i) \geq W.$$

Here polynomials p of degree at most k are considered as codewords. The intuitive meaning of weights is the measure of “confidence” that the “transmitted” p has $p(x_i) = y_i$. Thus, the goal is to find all codewords, for which the overall agreement is greater than the specified bound. *A priori* there are no requirements for the points to be distinct, in particular, different points can have the same x -coordinate.

The weighted polynomial reconstruction problem plays an important role in the list decoding ([9]) and soft decoding algorithms ([10]), also it is used for decoding of the codes that result from the concatenation of Reed–Solomon with smaller inner codes ([8]). All these decoding procedures inherit the complexity of Problem 2, thus, it is an interesting and important goal to design fast solutions for this problem.

It was shown in a series of works by Sudan [14] and Guruswami and Sudan [9] that Problem 2 can be solved in time polynomial in the sum of the weights provided

$$W > \sqrt{k \sum_{i=1}^N \text{wt}^2(x_i, y_i)}$$

(this in particular implies that there can be at most polynomially many solutions). The main idea of their analysis is to consider the following interpolation subproblem.

Problem 3 (Weighted Curve Fitting):

INPUT: N points $(x_1, y_1), \dots, (x_N, y_N)$ with natural integer weights $\text{wt}(x_1, y_1), \dots, \text{wt}(x_N, y_N)$ and parameters k and D .
OUTPUT: A polynomial $Q(x, y)$ such that

1) For any point (x_i, y_i) , Q can be represented as a sum

$$Q = \sum_{t=0}^{\text{wt}(x_i, y_i)} g_{t,i}(x, y)(x - x_i)^t(y - y_i)^{\text{wt}(x_i, y_i) - t}. \quad (3)$$

2) $\deg_x(Q) + k \deg_y(Q) \leq D$.

In the case of continuous field, the first condition states that $Q(x, y)$ has a singularity of order $\text{wt}(x_i, y_i)$ in (x_i, y_i) or, equivalently, the first $\text{wt}(x_i, y_i)$ derivatives of Q are equal to 0 in the point (x_i, y_i) .

The original polynomial algorithm of Guruswami and Sudan deals with the weighted curve fitting by solving the corresponding homogeneous linear system. Following the publication of [14] and [9], several papers have been published that present fast implementations of that list decoding algorithm. In one of these papers, Feng [4] (followed by the journal publication of Feng and Giraud [5]) suggested a faster interpolation procedure for solving Problem 3 in time $(w \frac{n}{k})^{O(1)} n \log^3 n$. We construct an alternative fast algorithm that uses different ideas; in particular, we explicitly describe the Groebner basis for the ideal of all curves satisfying (3). Our upper bound is $\log n$ factor better than that of Feng and Giraud (see Section IV for a brief comparison of these approaches). The techniques we use can be considered a natural far-going generalization of the rational interpolation problem, which lies in the base of the original Welch–Berlekamp decoding algorithm [16] (in particular, the rational interpolation is a partial case of Problem 1). Our result is described by the following.

Theorem 1.2: There exists an algorithm that solves Problem 3 in time $(w \frac{n}{k})^{O(1)} M(n) \log n$, where w is the maximal overall weight assigned to a single line $x = x_i$ for all choices of i .

The next corollary follows the ideas of Roth and Ruckenstein [13], [4], and [5]. We discuss it in the Appendix.

Corollary 1.3: There exists an algorithm that solves Problem 2 in time $(w \frac{n}{k})^{O(1)} M(n) \log n$, where w is the maximal overall weight assigned to a single line $x = x_i$ for all choices of i , provided

$$W > \sqrt{k \sum_{i=1}^N \text{wt}^2(x_i, y_i)}.$$

The paper is organized in the following way. We prove Theorem 1.1 in Section II, whereas Section III is dedicated to the proof of Theorem 1.2. Finally, in the Appendix, we overview further steps in the weighted polynomial reconstruction after the curve with the required properties was found (how to infer Corollary 1.3 from Theorem 1.2).

II. SOLVING LINEAR DIOPHANTINE EQUATIONS

In this section, we design a fast algorithm solving Problem 1. First we reformulate the problem in terms of the shortest vector for the appropriate lattice. Here we exploit the classical lattice techniques for solving linear Diophantine equations and the problems of simultaneous approximation that goes back to the works of Dirichlet and Minkowski of the 19th Century.

Next, as in the original Knuth–Schönhage algorithm, we design a basic (slow) routine finding the shortest vector in Section II-A (in fact, in Knuth–Schönhage procedure such routine is the Euclid algorithm), and finally, in Section II-B, we optimize it to achieve the bounds of Theorem 1.1.

We will need the following basic facts about lattices over commutative rings. For a good survey on the topic we advise the reader to see [12].

Definition 2.1: Let R be an arbitrary commutative ring with 1. The lattice $\mathcal{L}_B$ generated by basis $B = \{\beta_1, \dots, \beta_\ell\}$, $\beta_i \in R^m$ is the set of all linear combinations

$$\mathcal{L}_B = \{f_1\beta_1 + f_2\beta_2 + \dots + f_\ell\beta_\ell : f_i \in R\}.$$

It is convenient to represent a basis as $m \times \ell$ -matrix $B = \{\beta_i\}$, that contains the vectors β_i as columns, then the lattice is described as $\mathcal{L}_B = \{B \cdot y : y \in R^\ell\}$.

Definition 2.2: Consider the space $R^{\ell \times \ell}$ of matrices with coefficients in R . A matrix M is *unimodular* ($M \in U_\ell(R)$) iff M can be represented as a product of elementary transvections $I + fe_{ij}$, where I is identity and e_{ij} contains a single 1 in the position (i, j) and zeros in the rest.

Two bases B_1 and B_2 are *equivalent* iff $B_1 = B_2 \cdot U$, for some $U \in U_\ell(R)$. In other words, they are equivalent if B_2 can be transformed to B_1 by elementary operations on the columns.

Since $U_\ell(R)$ is a group, the equivalence relation is well defined, equivalent bases generate the same lattice. Let us now

confine ourselves to the ring of polynomials $R = \mathbf{F}[x]$. We generalize the degree of a polynomial for a vector of polynomials and a set of vectors.

Definition 2.3: For a vector $\beta \in \mathbf{F}[x]^m$ denote its j th coordinate by $\beta(j)$. Let

$$\deg \beta = \max_{j=[1..m]} \deg \beta(j).$$

For a basis $B = \{\beta_1, \beta_2, \dots, \beta_\ell\}$ let $\deg B = \sum_{i=1}^\ell \deg \beta_i$.

Definition 2.4: An element $v \in \mathcal{L}$ is the *shortest vector* iff it is nonzero and for any other nonzero $v' \in \mathcal{L}$, $\deg v \leq \deg v'$.

Let us now look at Problem 1 in the light of this definition. It is easy to see that $d_1, \dots, d_m$ in (1) are the degrees of an arbitrary vector in the lattice generated by P_{ij} as a basis. Thus, Problem 1 can be reformulated in the following equivalent form.

Problem 4: Given a basis $B \in (\mathbf{F}[x]^{m \times \ell})$ find any shortest vector in $\mathcal{L}_B$.

There is a subtle point that in (1) we require the vector in the lattice to have degree strictly greater than 0, while in Problem 4 any nonzero solution qualifies. This difficulty can be avoided by multiplying every P_{ij} by x .

A. Basic Algorithm Computing the Shortest Vector

Recall, that for a polynomial $f = a_1x^d + a_2x^{d-1} + \dots + a_{d+1}$, its *leading term* is the monomial of the highest degree: $\text{LT}(f) = a_1x^d$. In the next definition, we introduce a leading coordinate of a vector, and then define an important notion of the reduced basis.¹

Definition 2.5: For a vector β , define its *leading coordinate* as

$$\text{LC}(\beta) = \max\{j : \deg \beta(j) = \deg \beta\}$$

(the biggest index of polynomial of the maximal degree in β).

The basis $B = \{\beta_i\}$ is *reduced* iff for any pair of distinct nonzero vectors $\beta_1, \beta_2 \in B$ holds $\text{LC}(\beta_1) \neq \text{LC}(\beta_2)$.

If a basis is not reduced then one can try to reduce it by elementary operations on its columns. It can be done efficiently as we show in the following.

Proposition 2.1: For any basis, there exists an equivalent reduced one.

Proof: First we construct an algorithm, that either decreases the degree of a basis at least by one or outputs the reduced basis. Consider the algorithm depicted in Fig. 1.

Given a basis B it computes a unimodular matrix $R(B)$ s.t. either $\deg(B \cdot R(B)) < \deg B$ or $B \cdot R(B)$ is reduced. Informally, it does the following: if the basis is not reduced then one can find two columns that have polynomials of the highest degree in the same position j (line 1 of the algorithm). Let $\text{LT}(\beta_{i_1}(j)) = a_1x^{d_1}$, $\text{LT}(\beta_{i_2}(j)) = a_2x^{d_2}$, assume without loss of generality (w.l.o.g.) that $d_1 \geq d_2$. We subtract the i_2 th

¹After the publication of the preliminary version of this paper it was pointed out by Peter Buerger that this notion (as well as Proposition 2.3) was known earlier in the context of factoring polynomials over finite fields (see [6], [11], and the references cited therein).

Algorithm $R(B)$

Input: basis $B \in \mathbf{F}[x]^{m \times \ell}$

Output: matrix $R(B) \in U_\ell(\mathbf{F}[x])$ s.t. either $B \cdot R(B)$ is reduced or $\deg(B \cdot R(B)) < \deg B$.

```

begin
  let  $n = \deg B$ ;  $U = I$ 
  while  $\deg(B) = n$  do
    begin
      if  $B$  is reduced then return  $U$ 
      else find  $i_1 \neq i_2$  s.t.  $\text{LC}(\beta_{i_1}) = \text{LC}(\beta_{i_2})$ 
      let  $j = \text{LC}(\beta_{i_1})$ 
      if  $\deg \beta_{i_1} < \deg \beta_{i_2}$  then switch  $i_1$  and  $i_2$ 
      let  $f = \text{LT}(\beta_{i_1}[j]) / \text{LT}(\beta_{i_2}[j])$ 
      let  $T = I - f e_{i_2 i_1}$ 
      let  $U = U \cdot T$ 
      let  $B = B \cdot T$ 
    end
  return  $U$ 
end

```

Fig. 1. The elementary reduction operator.

Algorithm $R(B, t)$

Input: basis $B \in (\mathbf{F}[x])^{m \times \ell}$

Output: matrix $R(B, t) \in U_\ell(\mathbf{F}[x])$ s.t. either $B \cdot R(B, t)$ is reduced or $\deg(B \cdot R(B, t)) \leq \deg B - t$.

```

begin
  let  $n = \deg B$ ;  $U = I$ 
  while  $\deg(B \cdot U) > n - t$  do
    begin
      if  $B \cdot U$  is reduced then return  $U$ 
      else let  $U = U \cdot R(B \cdot U)$ 
    end
  return  $U$ 
end

```

Fig. 2. The iterative reduction operator.

column multiplied by $(a_1/a_2)x^{d_1-d_2}$ from the i_1 th column to decrease the degree of $\beta_{i_1}(j)$. We store this elementary column operation in U and continue the process until we either decrease the degree of B or completely reduce the basis. Note that at each step either the degree of the whole β_{i_1} decreases or the leading coordinate $\text{LC}(\beta_{i_1})$ becomes strictly less, thus, at some moment the process will be finished. It will be useful later to have the following estimate on the complexity of computing R .

Lemma 2.2: $R(b)$ does at most $O(m\ell(m + \ell))$ operations of polynomial addition and multiplication of the polynomial by a constant.

Proof: At each step, the algorithm decreases $\text{LC}(\beta_{i_1})$ for some i_1 . Clearly, the maximal possible number of steps is ℓm . Finally, to multiply the matrices U and B by the transvection $I - f e_{i_2 i_1}$ one can subtract the column i_2 multiplied by f from the column i_1 (this step takes $m + \ell$ operations). $\square$

Given an elementary reduction R one can define another algorithm depicted in Fig. 2 that reduces the degree of the basis at least by t (or outputs the reduced basis). At each step it checks whether the basis is reduced, and if not then it applies the operator R to decrease the degree further, until the degree decreases at least by t .

Finally, given a basis B it is sufficient to compute the matrix $R(B, \deg B)$ to get the reduced equivalent basis $B \cdot R(B, \deg B)$. Proposition 2.1 is proved. $\square$

Proposition 2.3: Assume that B is a reduced basis. Let β be any nonzero vector of the minimal degree in B . Then β is the shortest vector in $\mathcal{L}_B$.

Proof: For the sake of contradiction assume that there exist $f_1, \dots, f_k$ s.t.

$$\deg \left(\sum_{i=1}^{\ell} f_i \beta_i \right) < \deg \beta. \quad (4)$$

Notice, that $\text{LC}(f_i \beta_i) = \text{LC}(\beta_i)$ for any nonzero f_i . Let us consider the vector $f_i \beta_i$ in the sum (4) of the maximal degree, if there are several such vectors choose one with the maximal leading coordinate (it can be done in a unique way because B is reduced). Denote $j = \text{LC}(f_i \beta_i)$. We claim that the leading term of $f_i \beta_i(j)$ does not cancel in the sum (4). Indeed, $\text{LT}(f_i \beta_i(j))$ can be canceled only by another vector $f_{i'} \beta_{i'}$ of the maximal degree. But this cannot happen as we have chosen the vector of the maximal degree with the maximal leading coordinate. The proposition follows. $\square$

One can observe that algorithm $R(B, t)$ already solves Problem 4. However, we are interested in faster solutions, for that we apply a trick similar to that of the original Knuth–Schönhage algorithm.

B. Improved Algorithm Computing the Shortest Vector

One of the main concepts of Knuth–Schönhage gcd algorithm is the following definition (we allowed ourselves a small modification to adjust it for our purposes).

Definition 2.6: For a polynomial P and an integer t , the *accuracy t approximation* of P is given by the formula

$$P|_t = \pi \left(P x^{t - \deg(P) - 1} \right) x^{\deg(P) + 1 - t}$$

where $\pi: \mathbf{F}[x, x^{-1}] \rightarrow \mathbf{F}[x]$ is a linear map defined by $x^k \mapsto x^k$ for $k \geq 0$ and $x^k \mapsto 0$ for $k < 0$. Two polynomials P_1 and P_2 are equivalent up to accuracy t ($P_1 \stackrel{t}{\sim} P_2$) iff $(P_1)|_t = (P_2)|_t$.

Informally, the accuracy t approximation of a polynomial is the sum of t highest terms

$$(a_1 x^d + a_2 x^{d-1} + \dots + a_{d+1})|_t = a_1 x^d + \dots + a_t x^{d-t+1}.$$

We generalize this definition for vectors of polynomials.

Definition 2.7: For $\beta \in \mathbf{F}^m$ let

$$\beta|_t = \pi \left(\beta \cdot x^{t - \deg(\beta) - 1} \right) x^{\deg(\beta) + 1 - t}$$

where vector β is multiplied by x^d and rounded up by $\pi(\cdot)$ independently for each coordinate. For $B \in \mathbf{F}^{m \times \ell}$ let

$$B|_t = \{(\beta_i)|_t\}.$$

The accuracy relation is extended in the straightforward way

$$\begin{aligned} \beta_1 \stackrel{t}{\sim} \beta_2 &\iff (\beta_1)|_t = (\beta_2)|_t \\ B_1 \stackrel{t}{\sim} B_2 &\iff (B_1)|_t = (B_2)|_t. \end{aligned}$$

Lemma 2.4:

1) If $\beta \stackrel{t}{\sim} \beta'$ then $x^l \beta \stackrel{t}{\sim} x^l \beta'$.

2) If $\beta_1 \stackrel{t}{\sim} \beta_1', \beta_2 \stackrel{t}{\sim} \beta_2'$, and $\deg(\beta_1) = \deg(\beta_2) = d$ then

$$\beta_1 + \beta_2 \stackrel{t}{\sim} \beta_1' + \beta_2'$$

where $\hat{t} = t - (d - \deg(\beta_1 + \beta_2))$.

Proof: The first claim is checked independently for each coordinate in the straightforward way.

Let us prove the second claim; we can assume w.l.o.g. that $\hat{t} \geq 1$ (otherwise it is trivial). In this case, we can write

$$\begin{aligned} \pi \left((\beta_1 + \beta_2) \cdot x^{\hat{t} - \deg(\beta_1 + \beta_2) - 1} \right) &= \pi \left((\beta_1 + \beta_2) x^{t-d-1} \right) \\ &= \pi \left((\beta_1' + \beta_2') x^{t-d-1} \right) \end{aligned}$$

which implies that the degrees of $\beta_1 + \beta_2$ and $\beta_1' + \beta_2'$ coincide and $\beta_1 + \beta_2 \stackrel{\hat{t}}{\sim} \beta_1' + \beta_2'$. $\square$

Definition 2.8: The *length* of a polynomial f is the minimal t s.t. $f(x) = a_1 x^n + a_2 x^{n-1} + \dots + a_t x^{n-t+1}$.

Proposition 2.5: For any field $\mathbf{F}$ two polynomials of length t can be multiplied in $M(t)$ arithmetic operations.

Theorem 2.1: There is an algorithm that finds the shortest vector in a lattice $\mathcal{L}_B$ in time $O((\ell + m)^4 M(n) \log n)$ where n is the maximal degree of polynomials in B .

Proof: First we need to establish several crucial properties of the operator $R(B, t)$.

Lemma 2.6: For any $t_1 < t$

$$R(B, t) = R[B, t_1] \cdot R[B \cdot R(B, t_1), t - (\deg B - \deg(B \cdot R(B, t_1)))].$$

Proof: Recall the construction of $R(B, t)$. The lemma states that in order to reduce the degree of a basis by t , the algorithm first reduces it by t_1 , computes the basis $\hat{B} = B \cdot R(B, t_1)$, and then reduces it further by $t - (\deg B - \deg \hat{B})$. Finally, it takes the composition of these reductions. $\square$

Lemma 2.7:

$$R(B, t) = R(B|_t, t). \quad (5)$$

Proof of Lemma 2.7: Let $B_1 = B$, $B_2 = B|_t$. Let us analyze the performance of $R(\cdot, t)$ on B_1 and B_2 , for that we run two copies of the algorithm in parallel. At the start, $B_1 \stackrel{t}{\sim} B_2$. By our definition of $R(B)$, the procedure $R(\cdot, t)$ will make the same elementary transformations on B_1 and B_2 as long as $B_1 \stackrel{t}{\sim} B_2$.

Assume that at some step of the algorithm $B_1 \stackrel{t}{\sim} B_2$. By Lemma 2.4, whenever R makes an elementary transformation $\beta_1 := \beta_1 + f \beta_2$ that does not change the degree of β_1 , the accuracy does not change either: $B_1 \stackrel{t}{\sim} B_2$. If the degree of B_1 decreases by d then the accuracy will decrease by the same d . It is left to notice that the algorithm $R(\cdot, t)$ stops after the degree is reduced by t . $\square$

Lemma 2.8: Any monomial in $R(B, t)_{ij}$ has the form $a x^{d_{ij} + e}$, where $d_{ij} = \deg \beta_i - \deg \beta_j$ and $|e| \leq t - 1$.

Proof of Lemma 2.8: We prove it by induction. The base follows readily from our definition of $R(B)$ (notice that $R(B)$ stops its computation immediately after the degree of

Algorithm $\hat{R}(B, t)$
Input: basis $B \in (\mathbf{F}[x])^{m \times \ell}$
Output: matrix $\hat{R}(B, t) \in U_\ell(\mathbf{F}[x])$ s.t. either $B \cdot \hat{R}(B, t)$ is reduced or $\deg(B \cdot \hat{R}(B, t)) \leq \deg B - t$.
begin
 let $B = B|_t$
 if $t = 1$ **then return** $R(B)$
 let $U_1 = \hat{R}(B, \lfloor t/2 \rfloor)$
 let $B_1 = B \cdot U_1$
 return $U_1 \cdot \hat{R}(B_1, t - \lfloor \deg B - \deg B_1 \rfloor)$
end

Fig. 3. The improved iterative reduction operator.

any vector β_j has decreased). Let us check the induction step. By Lemma 2.6 (for $t_1 = 1$)

$R(B, t) = R(B) \cdot R[B \cdot R(B), t - (\deg B - \deg(B \cdot R(B)))]$. Consider the matrix $\hat{B} = B \cdot R(B)$, assume that $\deg \hat{B} = \deg B - d$. Clearly, the degree of all columns in $\hat{B}$ coincides with those of B except one vector $\hat{\beta}_i$, whose degree is $\deg \beta_i - d$, hence, for all i, j the difference $|\hat{d}_{ij} - d_{ij}| \leq d$. By the induction hypothesis, any monomial in $R(\hat{B}, t - d)_{ij}$ has the form $ax^{\hat{d}_{ij}+e}$, $|e| < t - d - 1$, while any monomial in $R(B)_{ij}$ has the form $bx^{d_{ij}}$.

Finally, any monomial in the product

$$(R(B) \cdot R(\hat{B}, t - d))_{ij}$$

comes from the polynomial

$$R(B)_{ik} R(\hat{B}, t - d)_{kj}$$

for some k and hence can be written as the sum of terms

$$bx^{d_{ik}} \cdot ax^{\hat{d}_{kj}+e} = bax^{d_{ik}} \cdot x^{d_{kj}} \cdot x^{\hat{d}_{kj}-d_{kj}+e} = bax^{d_{ij}+e'}$$

where

$$|e'| = |d_{kj} - \hat{d}_{kj} + e| \leq d + (t - d - 1) = t - 1. \quad \square$$

Now we are ready to construct a faster algorithm to compute $R(B, t)$, depicted in Fig. 3.

The behavior of $\hat{R}(B, t)$ is described by the following two lemmas.

Lemma 2.9 (Correctness):

$$\hat{R}(B, t) = R(B, t).$$

Proof: We prove it by induction. If $t = 1$ then the equality is obvious. If $t > 1$ then

$$\begin{aligned} \hat{R}(B, t) &= \hat{R}[B|_t, \lfloor t/2 \rfloor] \cdot \hat{R}\left[B|_t \cdot \hat{R}(B|_t, \lfloor t/2 \rfloor), \right. \\ &\quad \left. t - (\deg B|_t - \deg(B|_t \cdot \hat{R}(B|_t, \lfloor t/2 \rfloor))) \right] \\ &= (\text{induction hypothesis}) \\ &= R[B|_t, \lfloor t/2 \rfloor] \cdot R\left[B|_t \cdot R(B|_t, \lfloor t/2 \rfloor), \right. \\ &\quad \left. t - (\deg B|_t - \deg(B|_t \cdot R(B|_t, \lfloor t/2 \rfloor))) \right] \\ &= (\text{Lemma 2.6}) = R(B|_t, t) = (\text{Lemma 2.7}) \\ &= R(B, t). \quad \square \end{aligned}$$

Lemma 2.10: The running time of $\hat{R}(B, t)$ is bounded by $O((\ell + m)^3 M(t) \log t)$.

Proof: The proof is based on Lemma 2.8. In fact, the only reason why $\hat{R}(B, t)$ is more effective than $R(B, t)$ is that it deals with polynomials of smaller length.

Let us first estimate the number of calls to $R(B)$. Each time we apply this operator, the degree is decreased at least by one, thus there can be at most t calls that take time $O(\ell m(\ell + m)t)$. Indeed, by Lemma 2.2, the cost of each call is $O(\ell m(\ell + m))$ additions and multiplications of polynomials of length 1 (e.g., single monomials).

Let $f(t)$ be the running time of $\hat{R}(\cdot, t)$ without taking into account the time for $R(B)$. Then

$$f(t) \leq O(\ell^2(\ell + m)M(t)) + 2f(t/2)$$

where the first term is the price of multiplying two matrices that contain polynomials of length $2t$. As a corollary, $f(t) = O(\ell^2(\ell + m)M(t) \log t)$, and the lemma follows. $\square$

Now we finish the proof of Theorem 2.1. At the beginning, the basis B has degree at most ℓn . It takes $O((\ell + m)^4 M(n) \log n)$ operations to compute the matrix $R(B, \ell n)$. It is left to apply it to B to get the reduced equivalent basis $B \cdot R(B, \ell n)$. The theorem is proved. $\square$

Since Problems 1 and 4 are equivalent, this implies Theorem 1.1.

III. INTERPOLATION AND SOFT DECODING OF REED–SOLOMON CODES

A. Warm-Up: Interpolation of Singletons

Before considering the general interpolation problem we show how the techniques developed in the previous section help to solve the interpolation task that arises in the original algorithm of Sudan [14]: all weights equal 1 and all x_i are distinct.

Assume that $x_1, \dots, x_n \in \mathbf{F}$ are different points and the values $y_1, \dots, y_n \in \mathbf{F}$ are given. The goal is to find a polynomial $Q(x, y)$ which evaluates to zero on $(x_1, y_1), \dots, (x_n, y_n)$ s.t. $k \deg_y(Q) + \deg_x(Q) \leq D$, where k, D are given parameters.

Lemma 3.1: Consider the polynomials

$$Q_0(x) = \prod_{i=1}^n (x - x_i), \quad Q_1(x) = \sum_{i=1}^n y_i \prod_{j \neq i} \frac{(x - x_j)}{(x_i - x_j)}.$$

Any polynomial $Q(x, y)$ s.t. $\forall i \in [n] Q(x_i, y_i) = 0$ can be represented in the form $(y - Q_1(x))g(x, y) + Q_0(x)g_0(x)$, where $g_1(x, y), g_0(x) \in \mathbf{F}[x, y]$.

Proof: Consider any such polynomial Q and divide it on $y - Q_1(x)$: $Q(x, y) = (y - Q_1(x))g(x, y) + f(x)$. Since for all $i \in [n]$ $y_i - Q_1(x_i)$ equals 0, $f(x_i) = 0$ for every i . This means that f is divisible on $Q_0(x)$. $\square$

This lemma gives a way to find the solution Q . One may seek for Q in the form

$$Q = Q_0(x)\hat{g}(x) + (y - Q_1(x)) \left(\sum_{t=0}^{n/k} g_t(x)y^t \right).$$

Then the condition $k \deg_y(Q) + \deg_x(Q) \leq D$ corresponds to the following optimization system:

$$\begin{aligned} \deg(Q_0 \hat{g} - Q_1 g_0) &= d_0 \\ \deg(g_0 - Q_1 g_1) &= d_1 \\ &\dots \\ \deg(g_{n/k} - Q_1 g_{n/k-1}) &= d_{n/k} \\ \max(d_0, d_1 + k, d_2 + 2k, \dots, d_{n/k} + (n/k)k) &\leq D \\ \max(d_0, d_1 + k, d_2 + 2k, \dots, d_{n/k} + (n/k)k) &\neq 0. \end{aligned}$$

The i th row in this system corresponds to the x -degree of monomials with y -degree equal i . The system is reducible to (1) by multiplying the i th line by $x^{k(i-1)}$. Thus, by Theorem 1.1, it may be solved in time $(n/k)^{O(1)} M(n) \log n$. It is left to check that one may generate $Q_0(x)$ and $Q_1(x)$ fast.

Proposition 3.2: For every (not necessarily different) $x_1, \dots, x_n$, the polynomial $Q_0(x)$ is computable in time $O(n \log n)$. If $x_1, \dots, x_n$ are pairwise different then the polynomial $Q_1(x)$ is computable in time $O(M(n) \log n)$.

The proof of the first part of Proposition 3.2 may be found in [3, p. 54, Exercise 2.8] (because the coefficients of $Q_0(x)$ are exactly the elementary symmetric polynomials). The second part of Proposition 3.2 is described in [3, pp. 75–79].

B. Introducing Weights

In this subsection we prove Theorem 1.2. Let us briefly recall some basic notions in commutative algebra. Refer to [15] for a more complete treatment of the subject.

We will work in the ring $\mathbf{F}[x, y]$ of two-variable polynomials over $\mathbf{F}$. Polynomials consist of *terms*, a term (or monomial) is an expression $ax^{d_x}y^{d_y}$, $a \in \mathbf{F}$. Denote the set of all terms by T_{xy} . In order to study ideals of the ring of multivariate polynomials, it is convenient to define an ordering on the set of all terms. A total ordering $\preceq$ on T_{xy} is called *admissible* iff

- 1) $t_1 \preceq t_2 \Rightarrow \forall t \in T_{xy} \, tt_1 \preceq tt_2$;
- 2) $\forall t \in T_{xy} \, 1 \preceq t$.

In this subsection we consider the following family of admissible orderings. For a natural parameter k , let $(1, k)$ -weighted degree of term $t = ax^{d_x}y^{d_y}$ be $\deg_{(1,k)}(t) = d_x + kd_y$. For any k , fix an admissible ordering $\preceq_k$, s.t.

$$\deg_{(1,k)}(t_1) < \deg_{(1,k)}(t_2) \Rightarrow t_1 \prec_k t_2$$

(if both terms have the same $(1, k)$ -weighted degree then we compare them in lexicographical order). One can also define the ordering $\preceq_\infty$ that corresponds to infinite k

$$x^{d_x^1}y^{d_y^1} \prec_\infty x^{d_x^2}y^{d_y^2} \iff \begin{cases} d_y^1 < d_y^2 & \text{or} \\ d_y^1 = d_y^2 & \text{and } d_x^1 < d_x^2. \end{cases}$$

For a fixed ordering $\preceq$, the *leading term* $\text{LT}(P)$ of a polynomial P is the maximal term according to $\preceq$. For an ideal I , $\text{LT}(I)$ is the ideal generated by the leading terms of I

$$\text{LT}(I) = \text{Span}(\text{LT}(P) \mid P \in I).$$

A set of polynomials $Q_1, \dots, Q_t$ is called the *Groebner basis* for I iff each $Q_i \in I$ and

$$\text{Span}(\text{LT}(Q_1), \dots, \text{LT}(Q_t)) = \text{LT}(I).$$

The following simple statement is a useful description of the ideal in terms of its Groebner basis.

Statement 3.3: If $Q_1, \dots, Q_t$ is a Groebner basis for the ideal I then every polynomial $Q \in I$ can be written as

$$Q = \sum g_i Q_i$$

so that $\text{LT}(Q) = \text{LT}(g_i Q_i)$ for some $i \in [1..t]$.

Finally, we say that Q is the *minimal polynomial* for an ideal I , iff $Q \in I$, and $\text{LT}(Q) = \min\{\text{LT}(Q') \mid Q' \in I, Q' \neq 0\}$.

Let us now turn to the proof of Theorem 1.2. Assume that the points $(x_i, y_i) \in \mathbf{F}^2$ with weights $\text{wt}(x_i, y_i) \in \mathbf{N}$ are given. We are interested in the following ideal in $\mathbf{F}[x, y]$ which essentially corresponds to the set of all algebraic curves that go through points (x_i, y_i) at least $\text{wt}(x_i, y_i)$ times

$$I = \bigcap_{i=1}^N I_{\text{wt}(x_i, y_i)}(x_i, y_i) \quad (6)$$

where

$$I_w(x_i, y_i) = \text{Span}((x - x_i)^d \cdot (y - y_i)^{w-d} \mid d \in [0..w]). \quad (7)$$

We can now reformulate the weighted curve fitting problem in terms of the minimal polynomial in I .

Problem 5: Given a tuple of N points $(x_i, y_i) \in \mathbf{F}^2$ with weights, and an integer k , find a minimal polynomial in I according to the ordering $\preceq_k$.

The rest of the subsection will be dedicated to solving this problem. Our first goal is to describe a Groebner basis for the ideal I according to $\preceq_\infty$. Given this basis, it will be easy to modify it to find the minimal polynomial for $\preceq_k$.

Definition 3.1: For a multiset S with $|S| = w$ we say that the enumeration $L = \{s_1, \dots, s_w\}$ of its elements is *balanced* iff it was generated by the following algorithm.

- Draw the most frequent element in S and remove it from S
- Repeat w times.

For a balanced list $L = \{s_1, \dots, s_w\}$, let the k -shift of L be $L^{(k)} = \{s_{k+1}, s_{k+2}, \dots, s_w\}$. Denote by $d(L)$ the maximal number of equal elements in the list L (which coincides with the multiplicity of s_1).

Definition 3.2: Let n be the number of different values for x_i , and w_i be the sum of all weights over the line $x = x_i$. Assume w.l.o.g. that the coordinates $x_1, \dots, x_n$ are different. Enumerate all points in every vertical line in the list

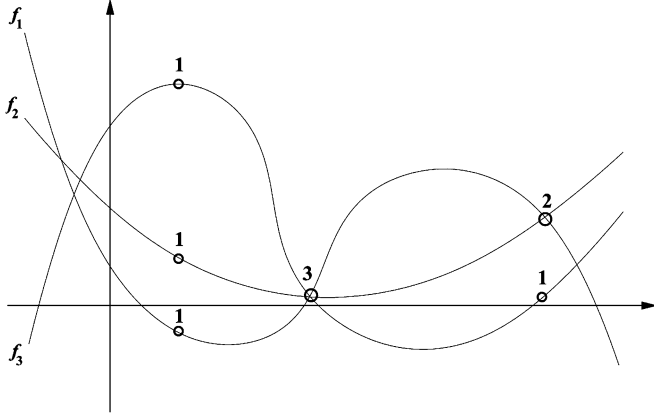
$$L_i = ((x_i, y_i^1), (x_i, y_i^2), \dots, (x_i, y_i^{w_i}))$$

so that

- each point (x_j, y_j) is represented exactly $\text{wt}(x_j, y_j)$ times in the list L_i s.t. $x_i = x_j$, $i \leq n$;
- the enumeration is balanced.

Let $w = \max_{i=1}^n w_i$ be the maximal weight among all vertical lines.

Theorem 3.1: Assume that $L_i = \{(x_i, y_i^j)\}$ is a balanced enumeration of the points, let $f_1(x), \dots, f_w(x)$ be a family of

Fig. 4. Construction of f_1, f_2, f_3 for $w = 3$.

univariate polynomials such that $\forall j \in [1..w]$ f_j is an interpolant given by $f_j(x_i) = y_i^j$ for all $i \in [1..n]$ (see Fig. 4). Let

$$Q_t(x, y) = \prod_{j=1}^t (y - f_j(x)) \prod_{i=1}^n (x - x_i)^{d_i(t)}$$

where $d_i(t) = d(L_i^{(t)})$ (in particular, $Q_0(x, y)$ is a univariate polynomial that depends on x). Then the family

$$\mathcal{G}(I) = \{Q_0(x, y), Q_1(x, y), \dots, Q_w(x, y)\} \quad (8)$$

is the Groebner basis for the ideal I according to $\preceq_\infty$.

Proof: First we have to check that the polynomials $Q_0, \dots, Q_w$ indeed belong to I . For that, fix some $t \in [0..w]$ and consider a point $(x_i, y_i^j) \in L_i$. Assume that d is the multiplicity of (x_i, y_i^j) in the shifted list $L_i^{(t)}$. The following is obvious in the case of continuous field (and can be checked in the straightforward way for an arbitrary field): the product $\prod_{j=1}^t (y - f_j(x))$ has a singularity of the order

$$\geq \text{wt}(x_i, y_i^j) - d \geq \text{wt}(x_i, y_i^j) - d_i(t)$$

in the point (x_i, y_i^j) , thus, $Q_t \in I_{\text{wt}(x_i, y_i)}(x_i, y_i)$.

It is left to show that the leading terms of $Q_0, \dots, Q_w$ generate the ideal $\text{LT}(I)$. For that, fix a polynomial $Q(x, y) \in I$ with $\deg_y(\text{LT}(Q)) = t$. If $t \geq w$ then

$$\text{LT}(Q) \in \text{Span}(\text{LT}(Q_w))$$

(because $d_i(w) = 0$ for any i), thus, it is sufficient to consider the case $t < w$.

Lemma 3.4: For $i \in [1..n]$, let $I(L_i)$ be the ideal (6) spanned on the points of L_i (e.g., on the points of the vertical line $x = x_i$). Then any polynomial $Q \in I(L_i)$ of y -degree t divides upon $(x - x_i)^{d_i(t)}$.

Proof: We use induction according to Definition 3.1. The base ($t = 0$) follows from Bezout theorem. To see the induction step, let us consider the point (x_i, y_i^1) (first in the list and of the maximal weight). Since $Q \in I_{\text{wt}(x_i, y_i^1)}(x_i, y_i^1)$ we can write it as

$$Q = (y - y_i^1) Q'(x, y) + (x - x_i)^{\text{wt}(y_i^1)} Q_0(x, y).$$

The second term in this sum belongs to $I(L_i)$ and satisfies the condition of the claim, thus, we can subtract it from Q .

Let us decrease the weight of y_i^1 by 1 and consider the resulting ideal $I' = I(L_i^{(1)})$. Notice that $\deg_y(Q') = t - 1$ and this polynomial belongs to I' . Clearly

$$d'_1(t - 1) = d\left(\left[L_i^{(1)}\right]^{(t-1)}\right) = d\left(L_i^{(t)}\right) = d_1(t)$$

and we can apply our induction hypothesis to show that Q' divides on $(x - x_i)^{d'_1(t-1)}$. The lemma follows. $\square$

Now we can easily finish the proof of the theorem. For any i , Q divides on $(x - x_i)^{d_i(t)}$, thus, $\text{LT}(Q) \in \text{Span}(\text{LT}(Q_t))$. The theorem follows. $\square$

Corollary 3.2: There is an algorithm that solves Problems 3 and 5 in time $(w \cdot \frac{n}{k})^{O(1)} M(n) \log n$, where w is the maximal overall weight assigned to a single line and n is the number of distinct lines.

Proof:

Lemma 3.5: For every $t \leq w$, Q_t may be generated in time $O(w^4 M(n) \log n)$.

Proof: By Proposition 3.2, the product $\prod_{i=1}^n (x - x_i)^{d_i(t)}$ may be computed in time $O(w n \log(w n))$, each interpolant $f_j(x)$ is computable in time $M(n) \log n$, finally, it takes at most t^3 of univariate polynomial multiplications to compute $\prod_{j=1}^t (y - f_j(x))$ each multiplication takes $M(w n)$ steps. $\square$

Consider the minimal $Q \in I$ according to $\preceq_k$ with $\deg_y(Q) = \ell$. We can assume w.l.o.g. that $\ell \leq \frac{w n}{k}$ (because the ideal contains a univariate polynomial of x -degree $w n$). By Statement 3.3, we can write

$$Q = \sum_{t \leq w} g_t(x, y) \cdot Q_t(x, y) \quad (9)$$

so that the highest term of the right part doesn't cancel out, in particular for every g_t , $\deg_y(g_t) \leq \ell$. Thus, one can represent g_t as the sum of $\ell + 1$ univariate polynomials $g_t = \sum_{j=0}^{\ell} g_t^j(x) y^j$. Let us consider each $g_t^j(x)$ as an unknown variable and write the optimization problem $\deg_x(Q) + k \deg_y(Q) \rightarrow \min$ in the form

$$\begin{aligned} \forall r \in [0.. \ell + w] \quad \deg\left(x^{kr} \pi_{y^r} \left(\sum g_t Q_t\right)\right) &= d_r \\ \max_{r=[0.. \ell + w]} (d_r) &\rightarrow \min \\ \max_{r=[0.. \ell + w]} (d_r) &\neq 0 \end{aligned} \quad (10)$$

where

$$\pi_{y^r} \left(\sum a_j(x) y^j\right) = a_r(x)$$

is the projection on y^r . Clearly, $\pi_{y^r}(\sum g_t Q_t)$ is a linear form over $(w + 1)(\ell + 1)$ unknown variables g_t^j that can be computed from the input in time $(w \ell)^{O(1)} M(n) \log n$. The degree d_r in (10) is exactly the sum $k \deg_y + \deg_x$ of the resulting polynomial. We have reduced the problem to the system (1), which can be solved by the algorithm of Section II. This solves Problem 3. In an analogous way, one can find the polynomial Q which is optimal for (10) subject to constraint $\deg_y(Q) \leq d_y$ for $d_y = 0, 1, \dots, \ell$ and choose the minimal polynomial according to $\preceq_k$ to solve Problem 5. $\square$

Algorithm *Roots*(Q, t)
Input: polynomial $Q(x, y)$, integer t
Output: The list of all solutions for $Q(x, y) = 0 \bmod x^t$
 represented in the form $G_1 \sqcup G_2 \sqcup \dots \sqcup G_\ell$ s.t. $G_i = \{y_i + gx^{d_i} | g \in \mathbf{F}[x]\}$

```

begin
  let  $Q = Q \bmod x^t$ 
  if  $t = 1$  then return all solutions of  $Q(0, y)$  using any factorization algorithm
    for univariate polynomials
  else let  $t_1 = \lfloor t/2 \rfloor$ 
  let  $G_1, \dots, G_\ell = \text{Roots}(Q, t_1)$ 
  for all  $i = 1.. \ell$  do
    begin
      let  $r_i = \max\{r_i \text{ s.t. } x^{r_i} | Q(x, y_i + x^{d_i} \hat{y})\}$ 
      let  $Q_i = Q(x, y_i + x^{d_i} \hat{y}) / x^{r_i}$ 
      let  $G_1^i, \dots, G_\ell^i = \text{Roots}(Q_i, t - r_i)$ 
    end
  return all non-empty lists  $G_i^j$ 
end

```

Fig. 5. Factorization of $Q(x, y)$.

Theorem 1.2 is proved.

IV. COMPARISON WITH THE ALGORITHM OF FENG AND GIRAUD

In this section, we briefly compare our algorithm with that of Feng and Giraud [5] which works in time $O(n \log^3 n)$. Their algorithm uses a different approach of interpolation which looks in the algebraic language as follows. For the pairs $(x_1, y_1), \dots, (x_n, y_n)$ denote by $I^{(i)}$ the ideal of all curves that go through the points $(x_1, y_1), \dots, (x_i, y_i)$. Then we get a chain of ideals $I^{(0)} \supset I^{(1)} \supset \dots \supset I^{(n)}$. For every i [5] iteratively construct a basis $\vec{Q}^{(i)}$ for $I^{(i)}$ using previously constructed bases. In order to perform the computation fast the divide-and-conquer paradigm is recursively used two times, hence, another $\log n$ factor in the running time.

APPENDIX POLYNOMIAL RECONSTRUCTION PROBLEM

In this appendix, we overview the further steps to reconstruct all “close” polynomials after the weighted curve fitting was done. For the sake of completeness we provide a sketch of the fast factorization algorithm described in [13], [4], [5].

Theorem 1.1 (Guruswami and Sudan [9]): Assume that Q is a solution of the weighted curve fitting with $D > \sqrt{k} \sum w_i^2$. Then for any polynomial p s.t. $\sum_{i: p(x_i)=y_i} w_i \geq D$ holds $Q(x, f(x)) \equiv 0$.

Thus, in order to construct the required list of polynomials it is sufficient to find all roots $y = f(x)$ of the equation $Q(x, y) = 0$.

Theorem 1.2: There exists an algorithm that given a bivariate polynomial $Q(x, y)$ finds all solutions $y = f(x)$ s.t. $Q(x, f(x)) = 0$ in time $\ell^{O(1)} n \log n$, where $n = \deg_x(Q)$ and $\ell = \deg_y(Q(0, y))$.

Proof: The following lemma is a corollary of Roth and Ruckenstein analysis in [13].

Lemma 1.1: Consider the equation

$$Q(x, y) = 0 \bmod x^t \quad (11)$$

where $y = f(x)$ is an unknown polynomial. Let $\ell = \deg_y(Q(0, y))$. Then there are at most ℓ nonintersecting sets

$$G_i = \{f_i(x) + gx^{d_i} | g \in \mathbf{F}[x]\}$$

s.t.

$$\{y | Q(x, y) = 0 \bmod x^t\} = G_1 \sqcup G_2 \sqcup \dots \sqcup G_k.$$

Proof: By induction on t . The base $t = 0$ is trivial. Consider the case $t > 0$. Let $\mathcal{R} = \{y_1, \dots, y_k\}$ be the superset of all roots of the univariate polynomial $Q(0, y) = 0$ (assume w.l.o.g. that it has exactly ℓ roots). Clearly, any solution of (11) has the form $y_i + x\hat{y}$, where $y_i \in \mathcal{R}$ and $\hat{y} \in \mathbf{F}[x]$. So consider any y_i of multiplicity m_i and write

$$\begin{aligned} Q(x, y_i + x\hat{y}) = 0 \bmod x^t &\iff \frac{Q(x, y_i + x\hat{y})}{x^{r_i}} \\ &= 0 \bmod x^{t-r_i} \end{aligned}$$

where $r_i \geq 1$ is the maximal coefficient s.t. $x^{r_i} | Q(x, y_i + x\hat{y})$. Denote $Q_i(x, \hat{y}) = Q(x, y_i + x\hat{y}) / x^{r_i}$. It is a crucial observation that $\deg_{\hat{y}}(Q_i(0, \hat{y})) \leq m_i$. To see this, write

$$Q(x, y) = (y - y_i)^{m_i} P_1(y) + x P_2(x, y)$$

so that $P_1(y_i) \neq 0$ and plug in $y = y_i + x\hat{y}$ to get

$$\begin{aligned} Q(x, \hat{y}) &= Q(x, y_i + x\hat{y}) \\ &= x^{m_i} \hat{y}^{m_i} P_1(y_i + x\hat{y}) + x P_2(x, y_i + x\hat{y}). \end{aligned}$$

Any term of $\hat{y}$ -degree greater than m_i has also x -degree greater than m_i in this polynomial. Thus, if $\deg_{\hat{y}} Q_i(0, \hat{y}) > m_i$ then $r_i > m_i$. However, this is impossible, since $Q(x, \hat{y})$ contains a term

$$P_1(y_i) x^{m_i} \hat{y}^{m_i}$$

that comes from the product $x^{m_i} \hat{y}^{m_i} P_1(y_i + x\hat{y})$ and does not cancel with any terms in $xP_2(x, y_i + x\hat{y})$, because the latter only contains terms with $\deg_{\hat{y}} < \deg_x$.

We have proved that $\deg_{\hat{y}} Q_i(0, \hat{y}) \leq m_i$ and thus we can use the induction hypothesis and construct at most m_i sets G_j^i that cover all solutions of (11) of the form $y = y_i + x\hat{y}$. After we combine these lists for all $y_i \in \mathcal{R}$ we get at most ℓ solution series. The lemma follows. $\square$

Now it is simple to finish the proof of Theorem 1.2. Consider an algorithm, depicted in Fig. 5. It uses a standard divide-and-conquer techniques to find all roots of the (11). For $t = 1$, it invokes any factoring procedure to find all roots of the univariate polynomial $Q(0, y)$ in polynomial time (for example, [2]). Let us estimate the time complexity of the algorithm, denote by $f(t, \ell)$ the time complexity of solving (11) provided $\ell = \deg_y(Q(0, y))$. We have a recursive bound

$$f(t, \ell) \leq \ell^{O(1)} t + f(t/2, \ell) + \max_{a_1+a_2+\dots+a_\ell=\ell} \sum_{i=1}^{\ell} f(t/2, a_i)$$

which gives the desired bound $f(t, \ell) = \ell^{O(1)} t \log t$. $\square$

ACKNOWLEDGMENT

The author wishes to thank Madhu Sudan, Piotr Indyk, and Venkatesan Guruswami for helpful discussions. Also, the author would like to thank Peter Buerger for pointing out references [6] and [11].

REFERENCES

[1] A. V. Aho, J. E. Hopcroft, and J. D. Ullman, *The Design and Analysis of Computer Algorithms*, ser. Computer Science and Information Processing. Reading, MA: Addison-Wesley, 1975, pp. 302–308.

[2] E. R. Berlekamp, “Factoring polynomials over finite fields,” *Bell Syst. Tech. J.*, vol. 46, pp. 1853–1859, 1967.

[3] P. B urger, M. Clausen, and M. Shokrollahi, *Algebraic Complexity Theory*, ser. Grundlehren der mathematischen Wissenschaften. Heidelberg, Germany: Springer-Verlag, 1996, vol. 315.

[4] G. L. Feng, “Two fast algorithms in the Sudan decoding procedure,” in *Proc. 37th Annu. Allerton Conf. Communication, Control and Computing*, Monticello, IL, Oct. 1999, pp. 545–554.

[5] G. L. Feng and X. Giraud, “Fast algorithms in Sudan decoding procedure for Reed–Solomon codes,” *IEEE Trans. Inf. Theory*, to be published.

[6] J. von zur Gathen, “Hensel and Newton methods in valuation rings,” *Math. Computation*, vol. 42, pp. 637–661, 1984.

[7] V. Guruswami, “List decoding of error-correcting codes,” Ph.D. dissertation, MIT, Cambridge, MA, 2001.

[8] V. Guruswami and P. Indyk, “Expander-based constructions of efficiently decodable codes,” in *Proc. 42nd IEEE Symp. Foundations of Computer Science (FOCS 2001)*, Las Vegas, NV, Oct. 2001, pp. 658–667.

[9] V. Guruswami and M. Sudan, “Improved decoding of Reed–Solomon and algebraic-geometric codes,” *IEEE Trans. Inf. Theory*, vol. 45, no. 6, pp. 1757–1767, Sep. 1999.

[10] R. Koetter and A. Vardy, “Algebraic soft-decision decoding of Reed–Solomon codes,” *IEEE Trans. Inf. Theory*, vol. 49, no. 11, pp. 2809–2825, Nov. 2003.

[11] A. K. Lenstra, “Factoring multivariate polynomials over finite fields,” *J. Comp. Sys. Sci.*, vol. 30, pp. 235–248, 1985.

[12] L. Lov  sz, “An algorithmic theory of numbers, graphs and convexity,” in *Conference Board of the Mathematical Sciences, Regional Conference Series*, 1986.

[13] R. Roth and G. Ruckenstein, “Efficient decoding of Reed–Solomon codes beyond half the minimum distance,” *IEEE Trans. Inf. Theory*, vol. 46, no. 1, pp. 246–257, Jan. 2000.

[14] M. Sudan, “Decoding of Reed–Solomon codes beyond the error-correction bound,” *J. Complexity*, vol. 13, no. 1, pp. 180–193, 1997.

[15] —, (1998) *Algebra and Computation*. [Online]. Available: <http://theory.lcs.mit.edu/~adhu>

[16] L. R. Welch and E. R. Berlekamp, “Error Correction of Algebraic Block Codes,” U.S. Patent 4,633,470, 1986.